

Unit 03: Requirement Engineering

CONTENTS

Objective

Introduction

- 3.1 Requirement Engineering
- 3.2 Requirements Engineering Tasks
- 3.3 Types of requirement
- 3.4 Requirement Negotiation
- 3.5 Requirements Validation
- 3.6 Requirements Management

Summary

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Reading

Objective

After studying this unit, you will be able to:

- Describe requirement engineering task
- Explain requirement management
- Explain requirement analysis and stakeholder analysis

Introduction

Requirements define the "what" of a system rather than the "how." Understanding what the customer wants, analyzing the need, determining feasibility, negotiating a reasonable solution, clearly specifying the solution, validating the specifications, and managing the requirements as they are transformed into a working system are all covered by requirements engineering. As a result, requirements engineering is the systematic use of well-established ideas, methodologies, tools, and notations to characterize a proposed system's expected behavior and restrictions.

In the engineering design process, requirements engineering refers to the process of identifying, documenting, and managing requirements. Understanding what the customer wants, analysing the need, determining feasibility, negotiating a reasonable solution, clearly specifying the solution, validating the specifications, and managing the requirements as they are transformed into a working system are all covered by requirement engineering. As a result, requirement engineering is the systematic use of well-established ideas, methodologies, tools, and notation to specify a proposed system's expected behaviour and restrictions.

It is process to gather, defining, documenting, and maintaining the software requirements from client, analyze and document them is known as requirement engineering. Requirement engineering provides the appropriate mechanism to understand what the customer desires, analyzing the need, and assessing feasibility, negotiating a reasonable solution.

3.1 Requirement Engineering

The requirements engineering includes the following activities:

- Identification and documentation of customer and user's needs.
- Creation of a document describing the external behavior and associated constraints that will satisfy those needs.
- Analysis and validation of the requirements document to ensure consistency, completeness and feasibility.
- Evolution of needs.

The fundamental output of requirements engineering is the specification of requirements. It is a system requirement if it describes both hardware and software, and a software requirements specification if it solely provides software needs. The system is viewed as a black box in both of these scenarios.

Requirements Specification

A text, a graphical model, a prototype, or a combination of these can be used to specify requirements. Depending on the system being created, a standard template or a flexible methodology can be used to elicit system specifications.

As a result of system and requirements engineering, a system specification is created. It offers information about the hardware, software, and database, among other things. It outlines the function as well as the constraints that must be met while creating a system. It also provides information about the system's input and output.

Requirements Validation

During the validation process, the work product created as a result of requirements engineering is evaluated for quality. Validation of requirements guarantees that all requirements have been written accurately, that inconsistencies and errors have been identified and addressed, and that the work result meets the process, project, and product standards.

Although the requirements validation can be done in any way that leads to discovery of errors, the requirements can be examined against a checklist. Some of the checklist questions can be: Are the requirements clear or they can be misinterpreted?

- Is the source of the requirements identified?
- Has the final statement of requirement been verified against the source?
- Is the requirement quantitatively bounded?
- Is the requirement testable?
- Does the requirement violate domain constraints?
- These questions and their likes ensure that validation team does everything possible for a thorough review.

Requirements Management

Throughout the life of a computer-based system, requirements may alter. The operations that assist a project team in identifying, controlling, and tracking requirements and modifications to these requirements at any point during the project are referred to as requirements management. Identification is the first step in managing requirements. Traceability tables are developed once the criteria have been identified. Each criterion is linked to one or more aspects of the system or its surroundings in this table. They may be used as a requirements database to understand how a change in one requirement affects other components of the system under construction.

A Brigade of Design & Construction

Any engineering activity's eventual purpose is to provide some form of documentation. The design documentation is supplied to the manufacturing team when a design effort is completed. This is a different group of people with different abilities than the design team. The production team can proceed to produce the product if the design documentation actually represent a complete design. In fact, they can develop a large portion of the product without the help of the designers. After

evaluating the software development life cycle today, it appears that the source code listings are the only software documentation that appears to meet the criteria of an engineering design.

Software runs on computers. It is a sequence of ones and zeros. Software is not a program listing. A program listing is a document that represents a software design. Compilers, linkers, and interpreters actually build (manufacture) software. Software is very cheap to build. You just press a button. Software is expensive to design because it is complicated and all phases of the development cycle are part of the design process. Programming is a design activity; a good software design process recognizes this and does not hesitate to code when coding makes sense.

More often than you might think, coding makes sense. Often, the process of coding the design reveals flaws and the need for additional design work. It is preferable if this occurs sooner rather than later. Testing and debugging are design tasks, and they are the engineering equivalents of design validation and refinement. They can't be taken advantage of. Because it is cheaper to design software and test it than to prove it, formal validation methods aren't very useful.

Think about it. When you are programming, you are doing detailed design. The manufacturing team for software is your compiler or interpreter. The source is the only complete specification of what the software will do.

Design is present whenever objects are developed for humans to utilize. Design can be done systematically or haphazardly, intentionally or unconsciously. However, when people create software – or any other product – they make decisions and build objects with the purpose of knowing what those objects will do and how they will be viewed and used..

The education of computer professionals has often concentrated on the understanding of computational mechanisms, and on engineering methods that are intended to ensure that the mechanisms behave as the programmer intends. The focus is on the objects being designed: the hardware and software. The primary concern is to implement a specified functionality efficiently.

When software engineers or programmers say that a piece of software works, they typically mean that it is robust, is reliable, and meets its functional specification. These concerns are indeed important. Any designer who ignores them does so at the risk of disaster.

But this inward-looking perspective, with its focus on function and construction, is one-sided. To design software that really works, we need to move from a constructor's-eye view to a designer's-eye view, taking the system, the users, and the context all together as a starting point.

3.2 Requirements Engineering Tasks

The processes used for RE vary widely depending on the application domain, the people involved and the organization developing the requirements.

However, there are a number of generic activities common to most processes:

Inception

Requirements elicitation

Negotiation

Requirements specification

Requirements validation

Requirement Management

Requirements Reviews/Inspections

Regular reviews should be held while the requirements definition is being formulated. Both client and contractor staff should be involved in reviews. (Stakeholders) Reviews may be formal (with completed documents) or informal. Good communications between developers, customers and users can resolve problems at an early stage.

Review Check-list

- Verifiability: Is the requirement realistically testable?
- Comprehensibility: Is the requirement properly understood?
- Traceability: Is the origin of the requirement clearly stated?

- Adaptability: Can the requirement be changed with minimum impact on other Notes requirements? (Especially when change is anticipated!)

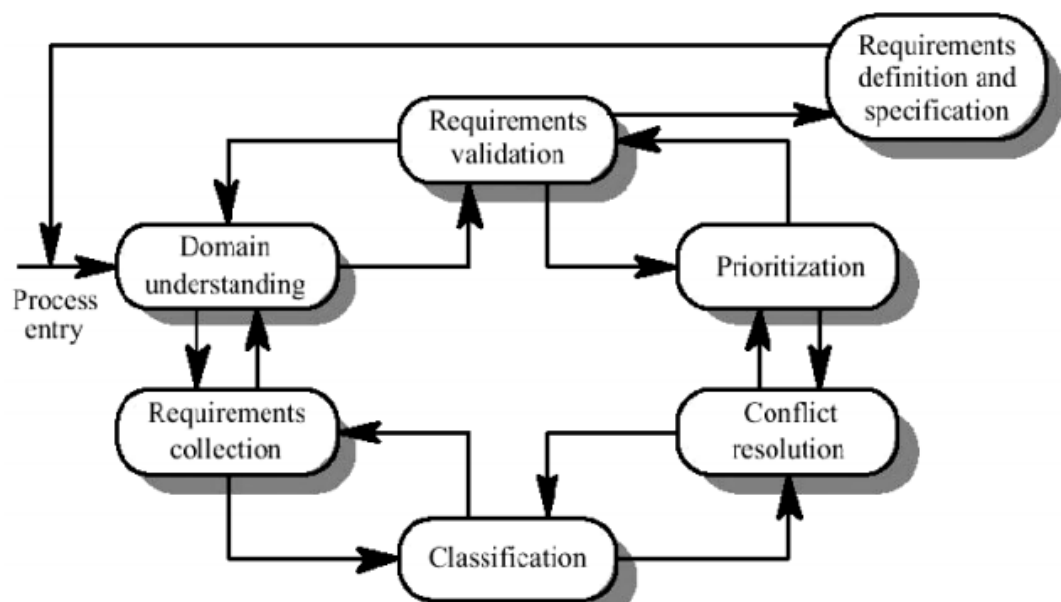
Elicitation

Involves working with customers to learn about the application domain, the services needed and the system's operational constraints. May also involve end-users, managers, maintenance personnel, domain experts, trade unions, etc. (That is, any stakeholders.)

Problems of Elicitation and Analysis

- Getting all, and only, the right people involved.
- Stakeholders often don't know what they really want ("I'll know when I see it").
- Stakeholders express requirements in their own terms.
- Stakeholders may have conflicting requirements.
- Requirements change during the analysis process.
- Organizational and political factors may influence the system requirements.

The Elicitation and Analysis Process



Viewpoint-oriented Elicitation

Stakeholders represent different ways of looking at a problem (viewpoints). A multi-perspective analysis is important, as there is no single correct way to analyze system requirements, provides a natural way to structure the elicitation process.

Types of Viewpoints: Data sources or sinks: Viewpoints are responsible for producing or consuming data. Analysis involves checking that assumptions about sources and sinks are valid.

Representation frameworks: Viewpoints represented by different system models (i.e., dataflow, ER, finite state machine, etc.). Each model yields different insights into the system.

Receivers of services: Viewpoints are external to the system and receive services from it, Natural to think of end-users as external service receivers.

Method-based RE

"Structured methods" to elicit, analyse, and document requirements.

Examples include:

Ross' Structured Analysis (SA),

Volere Requirements Process (www.volere.co.uk).

Knowledge Acquisition and Sharing for Requirement Engineering (KARE) (www.kare.org),

Somerville's Viewpoint-Oriented Requirements Definition (VORD), and The baut's Scenario-Based Requirements Engineering (SBRE)~

Scenarios

Depict examples or scripts of possible system behavior. People often relate to these more readily than to abstract statements of requirements, particularly useful in dealing with fragmentary, incomplete, or conflicting requirements.

Scenario Descriptions

System state at the beginning of the scenario, Sequence of events for a specific case of some generic task the system is required to accomplish. Any relevant concurrent activities System state at the completion of the scenario.

Scenario-Based Requirements Engineering (SBRE)

Marcel support environment allows rapid construction of an operational specification of the desired system and its environment. Based on a forward chaining rule-based language, an interpreter executes the specification to produce natural language based scenarios of system behavior.

UML Use-cases and Sequence Diagrams

Use-cases are a graphical notation for representing abstract scenarios in the UML. They identify the actors in an interaction and describe the interaction itself. A set of use-cases should describe all types of interactions with the system. Sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing.

Social and Organizational Factors

All software systems are used in a social and organizational context. This can influence or even dominate the system requirements. Good analysts must be sensitive to these factors, but there is currently no systematic way to tackle their analysis.

Ethnography

Social scientist spends considerable time observing and analyzing how people actually work. People do not have to explain or articulate what they do. Social and organizational factors of importance may be observed. Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

Focused Ethnography

Developed during a project studying the air traffic control process Combines ethnography with prototyping. Prototype development raises issues, which focus the ethnographic analysis. Problem with ethnography alone: it studies existing practices, which may not be relevant when a new system is put into place.

3.3 Types of requirement

User requirements

Statements in natural language plus diagrams of the services the system provides and its operational constraints Written for customers.

System requirements

A structured document setting out detailed descriptions of the system's functions, services and operational constraints, Defines what should be implemented so may be part of a contract between client and contractor.

Functional requirements

Statements of services the system should provide how the system should react to particular inputs and how the system should behave in particular situations. May state what the system should not do. Describe functionality or system services. Depend on the type of software, expected users and the type of system where the software is used. Functional user requirements may be high level

statements of what the system should do. Functional system requirements should describe the system services in detail.

Non-functional requirement

Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc. Often apply to the system as a whole rather than individual features or services.

These define system properties and constraints e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc. Process requirements may also be specified mandating a particular IDE, programming language or development method.

Types of non-functional requirement

Product requirement

Efficiency	Dependability
Security	Usability

Organizational requirement

Operational	Development
-------------	-------------

External requirements

Regulatory	Ethical
------------	---------

Non-functional classifications

Product requirements

Requirements which specify that the delivered product must behave in particular way e.g. execution speed reliability, etc.

Organizational requirements

Requirements which are a consequence of organizational policies and procedures e.g. process standards used, implementation requirements, etc.

External requirements

Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

3.4 Requirement Negotiation

After they have been collected, requirements must be analyzed to obtain a satisfactory understanding of the customer's need and negotiated to establish an agreed set of consistent (unambiguous, correct, complete, etc.) requirements.

The requirements negotiation process is expensive and time-consuming, requiring very experienced personnel exercising considerable judgment. This is exacerbated by the fact that the requirements negotiation process is not sufficiently structured that an automated requirements negotiation methodology would be appropriate.

Requirement Specification

For most engineering professions, the term "specification" refers to the assignment of numerical values or limits to a product's design goals. Typical physical systems have a relatively small number of such values. Typical software has a large number of requirements, and the emphasis is shared between performing the numerical quantification and managing the complexity of interaction among the large number of requirements.

For complex systems, particularly those involving substantial non-software components, as many as three different types of documents are produced: system definition, system requirements, and software requirements. For simple software products, only the third of these is required.

There are some approaches to requirements specification:

- Natural language
- Structured natural language
- Design description language
- Requirements specification language
- Graphical notation
- Formal specification

3.5 Requirements Validation

Concerned with whether or not the requirements define a system that the customer really wants.

Requirements error costs are high, so validation is very important. (Fixing a requirements error after delivery may cost up to 100 times that of fixing an error during implementation.)

Requirements Checking

Validity: Does the system provide the functions which best support the customer's needs?

Consistency: Are there any requirements conflicts?

Completeness: Are all functions required by the customer included?

Realism: Can the requirements be implemented given available budget and technology?

Verifiability: Can the requirements be tested?

Requirements Validation Techniques

Requirements reviews/inspections: systematic manual analysis of the requirements.

Prototyping: using an executable model of the system to check requirements.

Test-case generation: developing tests for requirements to check testability.

Automated consistency analysis: checking the consistency of a structured requirements description.

3.6 Requirements Management

Requirements management is the process of managing changing requirements during the requirements engineering process and system development. New requirements emerge during the process as business needs change and a better understanding of the system is developed. The priority of requirements from different viewpoints changes during the development process, the business and technical environment of the system changes during its development.

Enduring and Volatile Requirements

Enduring requirements: Stable requirements derived from the core activity of the customer organization. For example, a hospital will always have doctors, nurses, etc. May be derived from domain models, Volatile requirements: Requirements which change during development or when the system is in use. E.g., requirements derived from the latest health-care policy.

Classification of Requirements

Mutable requirements: those that change due to changes in the system's environment. Emergent requirements: those that emerge as understanding of the system develop. Consequential requirements: those that result from the introduction of the system. Compatibility requirements: those that depend on other systems or organizational processes.

Requirements Management Planning

During requirements management planning, you must decide on: Requirements identification: How requirements will be individually identified. A change management process: A process to be followed when analyzing the impact and costs of a requirements change. Traceability policies: The amount of information about requirements relationships that is maintained. CASE tool support: The tool support required to help manage requirements change.

Traceability

Traceability is concerned with the relationships between requirements, their sources, and the system design.

Source traceability – links from requirements to stakeholders who proposed these requirements

Requirements traceability – links between dependent requirements.

Design traceability – links from the requirements to the design.

CASE Tool Support

Requirements storage – requirements should be managed in a secure, managed data store.

Change management – the process of change management is a workflow process whose stages can be defined and information flow between the stages partially automated.

Traceability management – automated discovery and documentation of relationships between requirements

Requirements change management should apply to all proposed changes to the requirements.

Principal Stages

Problem analysis – discuss identified requirements problem and propose specific changes.

Change analysis and costing – assess effects of change on other requirements.

Change implementation – modify requirements document and others to reflect change.

Summary

The requirements engineering process includes feasibility study, elicitation and analysis, specification, and validation.

Requirements analysis is an iterative process involving domain understanding, requirements collection, classification, structuring, prioritization and validation.

Systems have multiple stakeholders with different viewpoints and requirements. Social and organization factors influence system requirements.

Requirements validation is concerned with checks for validity, consistency, completeness, realism, and verifiability.

Business, organizational, and technical changes inevitably lead to changing requirements.

Requirements management involves careful planning and a change management process.

Keywords

KARE: Knowledge Acquisition and Sharing for Requirement Engineering

SA: Structured Analysis

SBRE: Scenario-Based Requirements Engineering

VORD: Viewpoint-Oriented Requirements Definition

Self Assessment

1. Software requirement gathering from client, analyze and document is known as

- A. Requirement Gathering
- B. Requirement Engineering
- C. Feasibility Study
- D. System Requirements Specification

2. The objective of requirement engineering is to develop and maintain_____ document.

- A. Feasibility Study
- B. System Requirements Specification
- C. Requirement Gathering
- D. Software Requirement Validation

3. Which one is a correct requirement type?

- A. Reliability
- B. Availability
- C. Usability
- D. All of the above

4. Which is not step of requirement engineering?

- A. Elicitation
- B. Documentation
- C. Design
- D. Analysis

5. What are the activities involve in requirement gathering?

- A. Interview
- B. Survey
- C. Group discussion
- D. All of above

6. Product requirement include_____

- A. Efficiency
- B. Operational
- C. Development
- D. All of above

7. What are the types of requirements?

- A. Functional
- B. Non -functional
- C. System
- D. All of above

8. Requirement Engineering process activities are_

- A. Feasibility Study
- B. Requirement Elicitation and Analysis
- C. Software Requirement Validation
- D. All of above

9. Economic Feasibility deals with.

- A. Technical activity
- B. Operational activity
- C. Cost activity
- D. None of above

10. Requirement Elicitation activities are_____

- A. Organizing Requirements
- B. Negotiation & discussion
- C. Documentation
- D. All of the above

11. Which is not Requirement Elicitation Techniques?

- A. Surveys
- B. Questionnaires
- C. Design
- D. Task analysis

12. Requirements Validation Techniques are_____

- A. Requirements reviews/inspections
- B. Prototyping
- C. Test-case generation
- D. All of above

13. How the interviews held between two persons across the table is____

- A. Non-structured
- B. One-to-one
- C. Written
- D. Group

14. Which is not a feasibility study?

- A. Operational
- B. Finance
- C. Modular
- D. Social

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. B | 2. B | 3. D | 4. C | 5. D |
| 6. A | 7. D | 8. D | 9. C | 10. D |
| 11. C | 12. D | 13. B | 14. C | |

Review Questions

1. What are the current process problems and how would the system help with these?
2. Is new technology needed new skills?
3. What must be supported by the system and what had not be supported?
4. Requirements may change throughout the life of a computer based system. Explain.
5. Requirements management is the process of managing changing requirements during the requirements engineering process and system development. Discuss.
6. What is significance of requirement engineering?



Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, Second Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering a Practioner's Approach, 5th edition, Tata McGraw
- Hill Higher education.
- Sommerville, Software Engineering, Sixth edition, Pearson Education.



- <http://ksi.cpsc.ucalgary.ca/KAW/KAW96/herlea/FINAL.html>
- http://en.wikipedia.org/wiki/Requirements_analysis
- <https://www.javatpoint.com/>